Task 1 — Research Document

Design and Planning of a RESTful API for a Centralized Learning Management System (LMS)

Integrating Web and Mobile Platforms

Introduction

The rapid growth of online education platforms has increased the demand for scalable, secure, and interoperable systems that can operate across multiple platforms and services. A Learning Management System (LMS) is a software application designed to manage, deliver, and track educational content, assignments, enrolments, timetables, and student performance. In previous semesters, a web-based LMS was developed using PHP and a relational database, alongside a mobile application created using Ionic. While both systems provide similar functionality, they currently operate independently and do not share a unified communication or data exchange mechanism.

To allow the LMS to function at its highest potential, the implementation of a centralized RESTful API is essential. Without an API, both platforms would continue to operate as isolated systems, resulting in duplicated logic, inconsistent data, limited scalability, and poor integration opportunities. By introducing a RESTful API, the LMS can establish a single source of truth for all academic and user data, ensuring that both the web application and mobile application access the same validated information in real time.

The API is also necessary to support cross-platform synchronisation, allowing actions performed on one platform — such as submitting an assignment, updating a timetable, or enrolling in a course — to be reflected immediately across all connected systems. In addition, a centralized API creates opportunities for future third-party integrations, such as analytics dashboards, push notification services, attendance systems, student portals, and external academic reporting tools. This makes the LMS more extensible and future-proof.

Furthermore, an API-based architecture improves maintainability and scalability by separating the front-end clients from the back-end business logic and data layer. This means that future updates to the mobile app, web app, or any external service can be implemented without fundamentally changing the core system. As a result, the API becomes the key infrastructure component that allows the LMS to evolve beyond a standalone application into a connected educational platform.

The purpose of this document is therefore to propose the design and planning of a RESTful API for the LMS, including its architecture, authorization model, security considerations, and the practical endpoint structures required to support web, mobile, and third-party integrations.

Part 1 — RESTful APIs (KU1)

A RESTful API (Representational State Transfer Application Programming Interface) is an architectural style used to design networked applications that communicate over HTTP. REST was formally introduced by Fielding as a set of architectural constraints for distributed systems (Fielding, 2000). (Fielding, 2000).

A RESTful API allows different software systems to communicate with each other by sending HTTP requests and receiving structured responses, typically in JSON format. In a client–server model, the client (such as a web application or mobile application) sends a request to a server, and the server processes that request and returns a response containing the requested data or confirmation of an action.

RESTful APIs are widely used because they are lightweight, flexible, and platform independent. They enable interoperability between different technologies, making them ideal for integrating web applications, mobile applications, and third-party systems. In the context of the proposed Learning Management System (LMS), a RESTful API will serve as a central communication layer connecting the web and mobile platforms while ensuring scalability and maintainability. (Fielding, 2000).

1.2 Core Principles of REST

RESTful APIs are based on a set of architectural principles that define how distributed systems should communicate. These principles ensure scalability, flexibility, and reliability in modern web systems. (Fielding, 2000).

Client–Server Architecture separates the user interface from data storage and business logic. Statelessness means each request contains all the information needed for processing. Cacheability improves performance by allowing selected responses to be temporarily stored. A Uniform Interface ensures consistent use of resources, HTTP methods, and endpoint design. A Layered System allows security, logging, caching, and gateway layers to be introduced without changing the client applications.

1.3 How Making a Request to an API Works (Request / Response Process)

Communication between a client and a RESTful API follows a structured request–response cycle using HTTP. The process begins when a client sends a request to an API endpoint with a method, URL, headers, optional query parameters, and an optional request body. The server then validates the request, checks authorization, performs business logic, and returns an HTTP response with a status code and JSON data. (MDN Web Docs, n.d.-a; MDN Web Docs, n.d.-b).

Example request:
GET /api/assignments

Example authenticated header:
Authorization: Bearer <access_token>

Example response:

```
[
  {
    "id": 1,
    "title": "Algebra Assignment",
    "status": "Published"
  }
]
```

1.4 HTTP Methods (GET, POST, PUT, PATCH, DELETE)

Method	Operation	Description
GET	Read	Retrieves a resource or a list of resources. Does not modify data. Safe and idempotent.
POST	Create	Creates a new resource. Requires a request body with the new resource data. Not idempotent.
PUT	Replace	Fully replaces an existing resource with new data provided in the request body. Idempotent.
PATCH	Update	Partially updates an existing resource. Only the fields in the request body are changed.
DELETE	Delete	Removes a resource from the server. The server returns confirmation upon success. Idempotent.

These methods make the API predictable, standardized, and easy to integrate across web, mobile, and third-party systems. (MDN Web Docs, n.d.-a).

1.5 Query String Parameters and Request Body

Query string parameters are used to filter or search results, such as:
GET /api/users?role=student

A request body is used to send structured JSON data when creating or updating records, such as:

```
{
  "title": "Algebra Assignment",
  "description": "Solve algebra equations",
```

"dueDate": "2026-06-01"
}

1.6 Proposed API Endpoints for the LMS

The following endpoint tables are designed to support both internal LMS clients and future third-party integrations such as student portals, analytics tools, and reporting services.

User Management Endpoints

Endpoint	Method	Query Params / Request Body	Auth Required	Scope Required	Success Status Code	Response Example
/api/users	GET	?role=student|teacher|admin	Yes	users:read	200 OK	[{"id":1,"name":"John Doe","role":"student"}]
/api/users/{id}	GET	None	Yes	users:read	200 OK	{"id":1,"name":"John Doe","email":"john@example.com","role":"student"}
/api/users	POST	{"name":"...","email":"...","role":"...","password":"..."}	Yes	users:write	201 Created	{"message":"User created successfully","id":15}

Assignment Management Endpoints

Endpoint	Method	Query Params / Request Body	Auth Required	Scope Required	Success Status Code	Response Example
/api/assignments	GET	?courseId=3&status=published	Yes	assignments:read	200 OK	[{"id":1,"title":"Math Assignment","status":"Published"}]
/api/assignments/{id}	GET	None	Yes	assignments:read	200 OK	{"id":1,"title":"Math Assignment","description":"Solve algebra problems","dueDate":"2026-05-10"}
/api/assignments	POST	{"title":"...","description":"...","dueDate":"..."}	Yes	assignments:write	201 Cre	{"message":"Assignment created

					ated	successfully","id":9 }
/api/assignments/{id}	PUT	{"title":"...","description":"...","dueDate":"..."}	Yes	assignments:write	200 OK	{"message":"Assignment updated successfully"}
/api/assignments/{id}	DELETE	None	Yes	assignments:write	200 OK	{"message":"Assignment deleted successfully"}

Submission and Grading Endpoints

Endpoint	Method	Query Params / Request Body	Auth Required	Scope Required	Success Status Code	Response Example
/api/submissions	POST	{"assignmentId":1,"studentId":5,"fileUrl":"..."}	Yes	submissions:write	201 Created	{"message":"Submission uploaded successfully","id":22}
/api/submissions	GET	?studentId=5&assignmentId=10	Yes	submissions:read	200 OK	[{"id":22,"assignmentId":10,"studentId":5,"status":"Submitted"}]
/api/submissions/{id}/grade	PATCH	{"grade":85,"feedback":"Well done"}	Yes	submissions:write	200 OK	{"message":"Grade updated successfully"}

Course Catalogue, Enrolment, and Timetable Endpoints

Endpoint	Method	Query Params / Request Body	Auth Required	Scope Required	Success Status Code	Response Example
/api/courses	GET	?department=ICT&semester=2	Yes	courses:read	200 OK	[{"id":3,"title":"Web Development","credits":6}]
/api/enrol	POS	{"studentId":5,"cours	Yes	enrolments:	201	{"message":"Student

ments	T	eId":3}		write	Created	enrolled successfully"}
/api/timetable	GET	?studentId=5&week=2026-W18	Yes	timetable:read	200 OK	[{"day":"Monday","module":" Computer Science ","time":"09:00-11:00","room":"Lab 2"}]

1.7 Error Response Design and Validation Strategy

A well-designed API should provide meaningful and consistent error responses. For the proposed LMS API, all error responses will follow a standardized JSON structure.

Example:

```
{
  "error": "Validation failed",
  "status": 422,
  "details": {
    "dueDate": "Due date must be in the future"
  }
}
```

Status Code	Meaning	When Used
400 Bad Request	Invalid input	Missing or malformed request body fields.
401 Unauthorized	No/invalid token	Token missing, expired, or invalid.
403 Forbidden	Insufficient scope	Valid token but insufficient permissions.
404 Not Found	Resource not found	Requested resource does not exist.
409 Conflict	Duplicate entry	e.g. Student already enrolled in a course.
422 Unprocessable Entity	Validation failure	Business rule violation (e.g. past due date).
500 Internal Server Error	Server error	Unexpected database or server failure.

Validation rules will include required field checks, data type checks, format validation, business rules, and file restrictions. (MDN Web Docs, n.d.-b).

Part 2 — Authorisation: OAuth 2.0 (KU2)

OAuth 2.0 is an industry-standard authorization framework formally defined by the Internet Engineering Task Force in RFC 6749. It allows applications to access protected resources without exposing the user's password.

The key roles are the Resource Owner, Client, Authorization Server, and Resource Server. OAuth 2.0 is beneficial because it improves security, enables granular access control through scopes, supports third-party integrations, and allows token revocation without changing user passwords. (IETF, 2012).

2.3 Key Components of OAuth 2.0

Client ID and Client Secret

When a client application registers with the LMS authorisation server, it is issued a Client ID and a Client Secret. The Client ID is a public identifier for the application, while the Client Secret is a confidential value known only to the application and the authorisation server. Together, they authenticate the client application when it requests access tokens. The Client Secret must always be stored securely on the server side and must never be exposed in client-side code or public repositories.

Access Tokens

An access token is a credential issued by the authorisation server that the client includes in the Authorization header of every API request using the Bearer scheme. In the LMS API, access tokens will be implemented as JSON Web Tokens (JWTs) as defined in RFC 7519 (IETF, 2015). A JWT encodes the user's identity, role, granted scopes, and expiry time. Example usage:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Access tokens are intentionally short-lived (typically 15 to 60 minutes), limiting exposure if intercepted. When a token expires, a refresh token is used to obtain a new one without requiring the user to log in again.

Scopes

Scopes define the specific permissions granted to an access token. Rather than giving full API access, OAuth 2.0 restricts access to only what is needed. The following scopes are proposed for the LMS API:

Scope	Permission Granted
assignments:read	View assignments and their details.
assignments:write	Create, update, and delete assignments (teachers only).
submissions:read	View student submissions.
submissions:write	Submit assignments (students) or update grades (teachers).
courses:read	View course listings and details.
enrolments:read	View enrolment records.
enrolments:write	Enrol or unenrol students from courses.
users:read	View user profiles and lists.
users:write	Create or update user accounts (administrators only).
timetable:read	View student timetable data.
admin:all	Full administrative access. Restricted to admin role only.

Table : OAuth 2.0 Scopes for the LMS API

2.4 How OAuth 2.0 Will Be Implemented in the LMS API

The LMS API will implement two OAuth 2.0 grant flows depending on the type of client:

- **Authorization Code Grant with PKCE** will be used for user-facing clients such as the Ionic mobile application and the PHP web application. PKCE (Proof Key for Code Exchange) protects mobile clients that cannot securely store a Client Secret.
- **Client Credentials Grant** will be used for machine-to-machine communication, such as third-party analytics or reporting tools, where no user interaction is required.

The implementation follows these steps:

- **Step 1:** The client sends its Client ID and credentials to POST /api/auth/token, specifying the grant type and required scopes.
- **Step 2:** The authorisation server validates the credentials and returns a signed JWT access token and a refresh token.
- **Step 3:** The client includes the access token in the Authorization: Bearer header of every subsequent API request.
- **Step 4:** The LMS API validates the token signature, expiry time, and required scope before processing the request.

- **Step 5:** When the access token expires, the client sends the refresh token to POST /api/auth/refresh to receive a new access token without user interaction.

For machine-to-machine integrations such as analytics or reporting tools, the API will also support the Client Credentials Grant. This combined approach provides secure and scalable authorization for internal and third-party systems. (IETF, 2012).

Part 3 — Security Considerations: OWASP API Security Top 10 (KU3)

The Open Web Application Security Project (OWASP) is a globally recognised nonprofit foundation dedicated to improving software security. It produces free, community-driven documentation, tools, and standards that help organisations build and maintain secure software systems.

The OWASP API Security Top 10 is a widely recognised list of the most critical API security risks. The latest official edition is the OWASP API Security Top 10 (2023). It includes Broken Object Level Authorization, Broken Authentication, Broken Object Property Level Authorization, Unrestricted Resource Consumption, Broken Function Level Authorization, Unrestricted Access to Sensitive Business Flows, SSRF, Security Misconfiguration, Improper Inventory Management, and Unsafe Consumption of APIs. (OWASP, 2023).

3.2 The OWASP API Security Top 10 (2023)

Ref	Threat	Description
API1	Broken Object Level Authorization (BOLA)	APIs fail to verify that the requesting user is authorised to access a specific object. Attackers manipulate object IDs in requests to access other users' data. This is the most commonly exploited API vulnerability.
API2	Broken Authentication	Weak or incorrectly implemented authentication allows attackers to compromise tokens or assume other users' identities, gaining unauthorised access.
API3	Broken Object Property Level Authorization	APIs expose more data fields than necessary or allow modification of fields the user should not have access to, leading to data leakage or unauthorised changes.
API4	Unrestricted Resource Consumption	APIs without request rate limits, size limits, or pagination can be exploited to cause denial-of-service conditions or excessive infrastructure costs.
API5	Broken Function Level Authorization	Administrative or privileged functions are accessible to regular users due to missing role-based access controls, enabling privilege

		escalation.
API6	Unrestricted Access to Sensitive Business Flows	Attackers exploit legitimate API flows at scale (e.g. mass registrations or bulk submissions), bypassing intended business logic.
API7	Server-Side Request Forgery (SSRF)	The API fetches a remote resource based on user-supplied input without validating the URL, allowing attackers to make the server send requests to internal systems.
API8	Security Misconfiguration	Improper configuration of servers, frameworks, or cloud infrastructure, including overly permissive CORS policies, unnecessary HTTP methods, or default credentials.
API9	Improper Inventory Management	Outdated, undocumented, or unmonitored API versions expose systems to attacks because older versions often lack current security controls.
API10	Unsafe Consumption of APIs	The API blindly trusts data received from third-party APIs without proper validation, potentially leading to injection attacks or data integrity issues.

Table : OWASP API Security Top 10 (2023) — Source: owasp.org/API-Security

3.3 Proposed Mitigations for the LMS API

Based on the OWASP API Security Top 10 (2023), two of the most critical and directly relevant threats to the LMS API have been selected for detailed mitigation planning.

API1: Broken Object Level Authorization (BOLA)

BOLA is the most critical API vulnerability and is highly relevant to the LMS API, which manages individual student records, grades, and submissions. This vulnerability occurs when an API endpoint accepts a resource identifier from the client — such as a student ID or submission ID — and returns the data without verifying that the requesting user is actually authorized to access that specific object. For example, a student authenticated as user ID 42 could potentially manipulate a request to retrieve the grade or submission of user ID 43 simply by changing the ID in the URL.

Mitigation in the LMS API: Every endpoint that returns or modifies user-specific data will enforce object-level authorization checks on the server side. The API will extract the authenticated user's identity and role from the validated JWT access token and cross-reference it against the resource being requested. For example, a request to GET /api/submissions?studentId=5 will verify that the token subject matches the student ID in the query, or that the requester holds a teacher or admin scope. These checks will be implemented as reusable middleware applied consistently across all object-level endpoints. Automated tests will be written specifically to verify that users cannot access data belonging to other users.

API4: Unrestricted Resource Consumption

This threat is directly relevant to the LMS API because the platform will be accessed by a large number of students and teachers simultaneously, particularly during high-traffic periods such as assignment deadlines. Without proper controls, the API becomes vulnerable to denial-of-service (DoS) attacks or abuse by automated scripts, which could make the platform unavailable to legitimate users at critical times.

Mitigation in the LMS API: Multiple layers of resource consumption controls will be implemented. First, rate limiting will be enforced at the API gateway level, restricting each authenticated client to a configurable maximum number of requests per minute (e.g. 60 requests per minute for students, 120 for teachers and administrators). Clients exceeding this limit will receive a 429 Too Many Requests response with a Retry-After header. Second, request body size limits will be enforced — for example, file uploads for assignment submissions will be capped at a defined maximum size to prevent large payloads. Third, all list endpoints will require pagination with a configurable maximum page size, preventing any single request from returning thousands of records at once.

To mitigate Unrestricted Resource Consumption, the LMS API will implement rate limiting, request size limits, and pagination. These measures will help prevent abuse, denial-of-service conditions, and performance degradation during high-traffic academic periods. (OWASP, 2023).

Conclusion

This document has presented the research and planning for a RESTful API designed to serve as the central communication layer of a Centralized Learning Management System integrating a PHP web application and an Ionic mobile application.

In Part 1, the foundational principles of REST were examined in detail, covering client–server architecture, statelessness, cacheability, uniform interface, and layered systems. The HTTP request–response cycle and all five HTTP methods were explained with practical examples. Query string parameters and request bodies were illustrated in the context of the LMS. A comprehensive set of proposed endpoints was presented across four resource groups, with authentication requirements, scopes, and example responses for each. A standardised error response strategy and HTTP status code reference were also included to support robust client integration.

In Part 2, OAuth 2.0 was identified and described as the authorisation framework for the LMS API. The roles of Client IDs, Client Secrets, access tokens, and scopes were explained in detail. A concrete implementation plan was presented using both the Authorization Code Grant with PKCE for user-facing clients and the Client Credentials Grant for machine-to-machine integrations.

In Part 3, the OWASP API Security Top 10 (2023) was reviewed in full using official sources. Two critical threats — Broken Object Level Authorization (API1) and Unrestricted Resource Consumption (API4) — were selected for detailed mitigation planning, with specific strategies proposed including server-side authorisation middleware and multi-layered rate limiting.

A centralized RESTful API is essential for enabling the LMS to function at its highest potential. It provides a secure, scalable, and standards-compliant communication layer that supports internal clients, future third-party integrations, and long-term system growth.

References

Fielding, R.T. (2000) Architectural Styles and the Design of Network-based Software Architectures. Doctoral dissertation. University of California, Irvine. Available at: https://ics.uci.edu/~fielding/pubs/dissertation/top.htm (Accessed: 25 March 2026).

IETF (2012) RFC 6749: The OAuth 2.0 Authorization Framework. Available at: https://datatracker.ietf.org/doc/html/rfc6749 (Accessed: 25 March 2026).

IETF (2015) RFC 7519: JSON Web Token (JWT). Available at: https://datatracker.ietf.org/doc/html/rfc7519 (Accessed: 25 March 2026).

MDN Web Docs (n.d.-a) HTTP request methods. Available at: https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods (Accessed: 25 March 2026).

MDN Web Docs (n.d.-b) HTTP response status codes. Available at: https://developer.mozilla.org/en-US/docs/Web/HTTP/Status (Accessed: 25 March 2026).

OWASP (2023) OWASP API Security Top 10 – 2023. Available at: https://owasp.org/API-Security/editions/2023/en/0x00-header/ (Accessed: 25 March 2026).